Security Groups, SSH and RDP

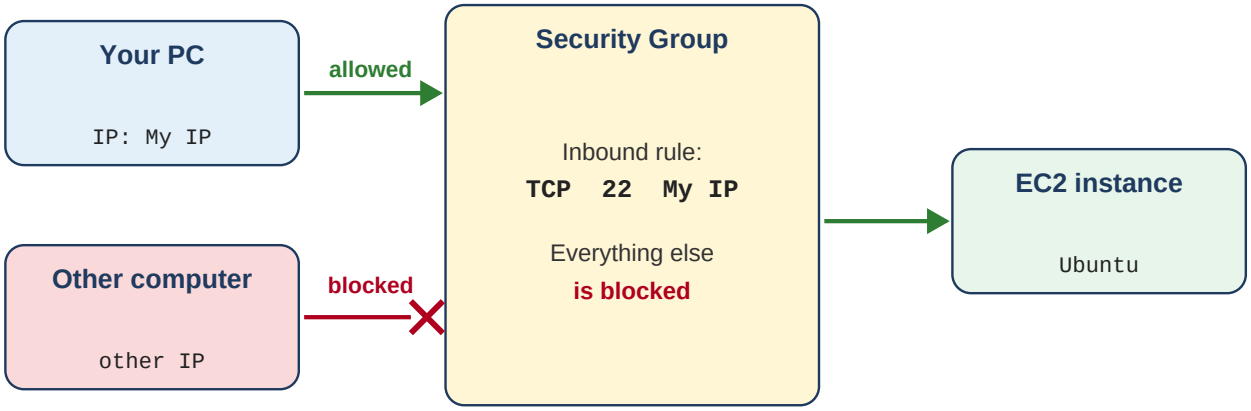
Controlling access to AWS instances and connecting to Linux and Windows servers: simple notes with steps and commands

Part	Topic	One-line idea
1	Security groups	A virtual firewall around your instance
2	Configure a security group	Create rules and attach them
3	SSH basics	Safe remote login for Linux
4	Connect with SSH	Commands to copy and paste
5	Create a Windows instance	Launch and get the Administrator password
6	Remote access: Linux vs Windows	SSH on port 22, RDP on port 3389
7	Troubleshooting ladder	Find out why you cannot connect
8	Quick quiz	Revise with answers

1. Security Groups

A **security group** is a **virtual firewall** for your AWS instances. It decides **which network traffic may reach your instance** and which traffic may leave it.

Easy picture: A security group is like the **security guard at the gate** of an apartment. The guard has a list: only people on the list can enter. Everyone else is stopped at the gate.



Term	Simple meaning
Inbound rule	Allows traffic **coming into** the instance.
Outbound rule	Allows traffic **leaving** the instance.
Protocol	The type of traffic: TCP, UDP or ICMP (ping).
Port range	The door number used by a service, e.g. 22 for SSH.
Source	Who may connect: an IP address range or another security group.

Term	Simple meaning
My IP	AWS fills in your current public IP as **x.x.x.x/32** (exactly one address).
0.0.0.0/0	Means **everyone on the Internet**. Dangerous for SSH and RDP.

How security groups behave

Behaviour	Meaning
Default inbound	Everything is **blocked** until you add an allow rule.
Default outbound	Everything is **allowed** (the instance can reach the Internet).
Allow rules only	You write only **allow** rules. There are no deny rules.
Stateful	If a request is allowed in, the reply is allowed out automatically (and the other way round).
Instant	Changes apply immediately. No restart needed.
Reusable	One security group can be attached to many instances, and one instance can have several.

Common ports

Service	Port	Protocol	Used for
SSH	22	TCP	Secure remote login to **Linux**
RDP	3389	TCP	Remote Desktop to **Windows**
HTTP	80	TCP	Websites (not encrypted)
HTTPS	443	TCP	Websites (encrypted)
MySQL / PostgreSQL	3306 / 5432	TCP	Databases

Example: a web server and its database

Security group	Inbound rules	Why
web-sg	SSH 22 from My IP HTTP 80 from 0.0.0.0/0 HTTPS 443 from 0.0.0.0/0	You manage it from your IP. The public may see the website.
db-sg	MySQL 3306 from **web-sg** (source = a security group)	Only the web servers can reach the database, nobody else.

Best practice: Give each role its own security group (web, database, admin). Use a **security group as the source** instead of IP addresses between your own servers.

2. Configure a Security Group (Step by Step)

1. **Open the EC2 dashboard**
   Left menu: Network and Security, then Security Groups
2. **Create security group**
   Give a name and description, choose the VPC
3. **Add inbound rule for SSH**
   Type SSH: TCP, port 22, source My IP
4. **Add inbound rule for RDP (Windows)**
   Type RDP: TCP, port 3389, source My IP
5. **Check outbound rules**
   Default allows all traffic. Keep it unless you need to restrict
6. **Create the security group**
   Click Create security group
7. **Attach it to the instance**
   At launch (Network settings), or later: Actions, Security, Change security groups

Remember: My IP changes. At home or on mobile data your public IP can change, and a rule made earlier then stops working. If you suddenly cannot connect, edit the rule and choose **My IP** again. You can check your current public IP with `curl -s ifconfig.me` (the networking notes explain public and private IPs).

3. SSH: Secure Shell Basics

SSH (Secure Shell) is a protocol that lets you **log in to a remote server and run commands over an encrypted connection**. Nobody on the network can read what you type.

Feature	Meaning
Secure communication	Everything is encrypted.
Authentication	Login with a password or, better, a **key pair**.
Port forwarding	Carries other traffic safely through the SSH connection (a tunnel).
File transfer	Tools **scp** and **rsync** copy files securely.
Sessions and commands	Run one command remotely, or keep sessions alive with tools like tmux.

SSH on AWS EC2: default settings

Setting	Value
Port	22 (must be allowed in the security group)
Authentication	Key pair: a **.pem** private key file made at launch

Setting	Value
Login user	ubuntu (Ubuntu), ec2-user (Amazon Linux, Red Hat), admin (Debian)

Easy picture: Lock and key: AWS puts the **lock** (public key) on the server. You keep the only **key** (the .pem file). Without the file nobody can log in, so never share it.

4. Connect with SSH: Commands

Your computer	SSH client
Linux, macOS	Built in: use the ssh command in the terminal.
Windows 10/11	Built in: use ssh in PowerShell. Or use PuTTY (convert .pem to .ppk with PuTTYgen).

Syntax	Meaning
chmod 400 KEYFILE.pem	Allow only you to read the key. SSH refuses open keys.
ssh -i KEYFILE.pem USER@PUBLIC_IP	Log in. -i = key file. USER = ubuntu for Ubuntu.
ssh -i KEYFILE.pem USER@PUBLIC_IP 'COMMAND'	Run one command remotely and come back.
scp -i KEYFILE.pem FILE USER@IP:FOLDER	Copy a file to the server.
scp -i KEYFILE.pem USER@IP:FILE .	Copy a file from the server to the current folder.
rsync -avz -e "ssh -i KEYFILE.pem" SRC USER@IP:DEST	Sync folders; sends only changes.
ssh -L LOCALPORT:localhost:PORT USER@IP	Port forwarding (tunnel).

COPY AND PASTE: Connect and run commands (replace the key name and PUBLIC_IP)

```
cd ~/Downloads
chmod 400 MyKey.pem
ssh -i MyKey.pem ubuntu@PUBLIC_IP
ssh -i MyKey.pem ubuntu@PUBLIC_IP 'uptime'
```

Command	What it does (in simple words)
chmod 400 MyKey.pem	Protect the key (permissions: Day 11).

Command	What it does (in simple words)
`ssh -i MyKey.pem ubuntu@PUBLIC_IP`	Start an interactive session. Type exit to leave.
`... 'uptime'`	Runs uptime on the server and returns to your own terminal.

Output (what students will see)

```
Are you sure you want to continue connecting (yes/no)? yes   <-- first time only
ubuntu@ip-172-31-5-20:~$               <-- inside the server
 10:30:15 up 2 days, 3:12, 1 user, load average: 0.00, 0.00, 0.00   <-- uptime
```

Copy files: scp and rsync

COPY AND PASTE: Upload and download files

```
echo "hello from my PC" > notes.txt
scp -i MyKey.pem notes.txt ubuntu@PUBLIC_IP:/home/ubuntu/
scp -i MyKey.pem ubuntu@PUBLIC_IP:/home/ubuntu/notes.txt ./copy.txt
mkdir -p project && echo test > project/a.txt
rsync -avz -e "ssh -i MyKey.pem" \
  ./project/ ubuntu@PUBLIC_IP:/home/ubuntu/project/
```

Command	What it does (in simple words)
`scp -i MyKey.pem notes.txt ubuntu@IP:/home/ubuntu/`	Upload notes.txt to the server's home folder.
`scp -i MyKey.pem ubuntu@IP:/home/ubuntu/notes.txt ./copy.txt`	Download it back with a new name.
`rsync -avz -e "ssh -i MyKey.pem" ./project/ ubuntu@IP:...`	Sync a folder: -a keeps permissions, -v shows files, -z compresses, -e chooses SSH with the key.

Output (what students will see)

```
notes.txt                 100%   19     0.9KB/s   00:00     <-- scp upload
sending incremental file list
a.txt                                        <-- rsync sent only this file
```

Shortcut: the SSH config file

Save the long command under a short name in `~/.ssh/config`, then connect with `ssh myec2`.

COPY AND PASTE: Create an SSH shortcut

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat >> ~/.ssh/config << 'EOF'
Host myec2
    HostName PUBLIC_IP
    User ubuntu
    IdentityFile ~/Downloads/MyKey.pem
EOF
chmod 600 ~/.ssh/config
ssh myec2
```

Port forwarding (tunnel) example

Suppose a web page runs on the server's port 80 but your security group does not allow port 80. The tunnel lets you see it **safely through SSH**.

COPY AND PASTE: Tunnel server port 80 to your PC port 8080

```
ssh -i MyKey.pem -L 8080:localhost:80 ubuntu@PUBLIC_IP
```

Keep that session open and browse to **http://localhost:8080** on your own PC. **-L 8080:localhost:80** means: my local port 8080 goes to port 80 on the server.

5. Create a Windows Instance

EC2 can run **Windows Server** for .NET applications or Windows-only tools. You connect with **RDP** (Remote Desktop Protocol), which shows the Windows desktop, instead of SSH.

1. **Open EC2 and click Launch instance**
   Check the region, give it a name
2. **Choose the AMI**
   Windows Server Base (your notes use 2019; newer versions like 2022 are also offered)
3. **Choose the instance type**
   t2.micro or t3.micro if Free tier eligible (Windows is slow with 1 GiB RAM)
4. **Create or select a key pair**
   Download the .pem file. It is needed to decrypt the password
5. **Network settings: security group**
   Add inbound rule RDP: TCP 3389, source My IP. Enable public IP
6. **Storage and launch**
   Keep the default storage, then Launch instance
7. **Wait, then get the Windows password**
   Instance, Connect, RDP client tab, Get password, upload the .pem, Decrypt password

Remember: The password is not ready immediately. Wait about **4 minutes** after launch. The default user is **Administrator**. The password is encrypted with your public key, so only your **.pem file** can decrypt it. Consider assigning an **Elastic IP** if you need a fixed address.

Connect with Remote Desktop

Your computer	Client	What to enter
Windows	Remote Desktop Connection (run mstsc)	Computer: PUBLIC_IP. User: Administrator. Password: the decrypted one.
macOS	Microsoft Remote Desktop app	Same details.
Linux	Remmina (or xfreerdp)	Same details. Example: xfreerdp /v:PUBLIC_IP /u:Administrator

6. Remote Access: Linux vs Windows

	Linux instance	Windows instance
Protocol	SSH	RDP
Port	22	3389
Security group rule	TCP 22 from My IP	TCP 3389 from My IP
Login user	ubuntu / ec2-user / admin	Administrator
Authentication	Key pair (.pem used directly)	Password, decrypted with the .pem
You see	A text terminal	The full Windows desktop
Client	ssh, PuTTY	Remote Desktop, Remmina

Memory trick: SSH = 22 = Linux (text). **RDP = 3389 = Windows** (desktop).

7. Troubleshooting Ladder: I Cannot Connect

Check these in order. Most problems are found in the first four.

#	Check	How
1	Is the instance **running** and 2/2 checks passed?	EC2, Instances: look at Instance state and Status check.
2	Is the **public IP** correct?	It changes after stop and start. Copy it again.
3	Does the **security group** allow your IP on 22 or 3389?	Edit inbound rules, set the source to My IP again.
4	Right **user name** and **key file**?	ubuntu for Ubuntu. Use the key chosen at launch.
5	Key permissions?	chmod 400 MyKey.pem (Linux/macOS).

#	Check	How
6	Is the SSH service or Windows firewall blocking?	Check inside the server (commands below).

COPY AND PASTE: Test whether the port is reachable from your PC

```
curl -s ifconfig.me; echo
IP=PUBLIC_IP
timeout 3 bash -c "</dev/tcp/$IP/22" 2>/dev/null && echo open || echo closed
```

Command	What it does (in simple words)
`curl -s ifconfig.me`	Shows your current public IP: compare it with the security group rule.
`IP=PUBLIC_IP`	Store the server address (replace PUBLIC_IP).
`timeout 3 bash -c "</dev/tcp/$IP/22" ...`	Tries to open port 22 for 3 seconds. Open means the network and security group allow it.

Output (examples)

```
49.36.10.5
open                    <-- the security group lets you in
closed                  <-- blocked, or the service is not running
```

If you can reach the server another way (for example the console's browser connection), check from **inside**:

COPY AND PASTE: Check SSH on the server

```
sudo systemctl status ssh
sudo ss -tlnp | grep ':22'
sudo ufw status
```

Command	What it does (in simple words)
`sudo systemctl status ssh`	Is the SSH service running? (On Red Hat the service is called sshd.)
`sudo ss -tlnp | grep ':22'`	Is something listening on port 22?
`sudo ufw status`	Is the Ubuntu firewall blocking? Inactive is normal on a new EC2 instance.

Symptom	Likely cause
Connection timed out	Security group, wrong IP, instance not running, or no public IP.
Permission denied (publickey)	Wrong user name or wrong key.
Unprotected private key / too open	Run chmod 400 on the key.
RDP: cannot connect	Port 3389 not allowed, no public IP, or still booting.

Symptom	Likely cause
RDP: password incorrect	Decrypt it again with the correct .pem file. User is Administrator.

Two layers of firewall

Layer	Where	Examples
Security group	Outside the instance, in AWS	Allow TCP 22 from My IP
Operating system firewall	Inside the instance	ufw (Ubuntu), Windows Firewall

Traffic must be allowed by **both** layers.

8. Best Practices, Quiz and Takeaways

- **Never open SSH (22) or RDP (3389) to 0.0.0.0/0.** Use My IP or your office range.
- Use **separate security groups** for different applications and roles.
- Keep **.pem keys private** and set them to chmod 400. Do not commit them to GitHub.
- Remove rules you no longer need, such as temporary open ports.
- For production, look at **EC2 Instance Connect** (browser SSH) and **Session Manager**, which need no open inbound ports.

Question	Answer
What is a security group?	A virtual firewall that controls inbound and outbound traffic of instances.
Inbound vs outbound rule?	Inbound = traffic into the instance. Outbound = traffic out.
What is the default of a new security group?	All inbound blocked, all outbound allowed.
What does stateful mean?	Replies to allowed traffic are allowed automatically.
Ports for SSH and RDP?	22 and 3389.
What does Source = My IP mean?	Only your current public IP (/32) can connect.
How do you get the Windows Administrator password?	Connect, RDP client, Get password, upload the .pem, Decrypt.
Which command copies a file to a server?	scp -i MyKey.pem file ubuntu@IP:/home/ubuntu/
You get Connection timed out. First check?	The security group rule and the public IP.

- Security groups control access to your instances, so configure them carefully.
- SSH (22) is for Linux and RDP (3389) is for Windows.
- Correct security groups give both access and safety.